

# Dokumentacja: Bootstrap

Bootstrap to framework CSS do szybkiego budowania responsywnych stron, formularzy, paneli, tabel, kart, nawigacji i komponentów interfejsu. Ten plik jest samodzielną dokumentacją Bootstrapa: można go czytać bez przewijania dokumentacji Reacta. Zawiera część ogólną, część o użyciu w React, tabele klas, praktyczne wzorce oraz złożone przykłady.

Dokument opisuje przede wszystkim Bootstrap 5.x. Wersja 5 nie wymaga jQuery. W React najczęściej korzysta się z klas CSS Bootstrapa, a komponenty interaktywne kontroluje się stanem Reacta albo importuje `bootstrap.bundle`, jeśli używa się natywnych mechanizmów `data-bs-*`.

## Spis treści

- 1. Bootstrap — podstawy i praca w React
- 2. Bootstrap bez Reacta: CDN i zwykły HTML
- 3. Responsywność i breakpointy
- 4. Utilities — klasy pomocnicze w praktyce
- 5. Formularze i walidacja
- 6. Komponenty Bootstrapa — katalog praktyczny
- 7. JavaScript Bootstrapa i React
- 8. Motywy, kolory, CSS variables i Sass
- 9. Dostępność i semantyka
- 10. Złożone przykłady praktyczne
- 11. Szybkie tabele referencyjne

## 1. Bootstrap — podstawy i praca w React

Bootstrap to framework CSS, czyli gotowy zestaw klas, komponentów i zasad układu strony. Zamiast pisać od zera style dla przycisków, formularzy, siatki, kart, alertów czy tabel, korzystasz z gotowych klas, np. `btn btn-primary`, `container`, `row`, `col-md-6`, `form-control`.

Bootstrap nie zastępuje Reacta. React odpowiada za logikę, stan, komponenty i renderowanie JSX. Bootstrap odpowiada za wygląd, responsywność i podstawowe zachowanie wybranych komponentów interfejsu.

W tym rozdziale pracujemy głównie z Bootstrapem 5, czyli wersją bez jQuery. W React używamy `className`, a nie `class`.

### 1.1. Czym jest Bootstrap i kiedy go używać

Bootstrap składa się z kilku dużych części:

| część      | DO CZEGO SŁUŻY                                                                                                 |
|------------|----------------------------------------------------------------------------------------------------------------|
| Layout     | <code>container</code> , <code>row</code> , <code>col</code> , breakpointy, responsywność                      |
| Utilities  | szybkie klasy typu <code>mt-3</code> , <code>d-flex</code> , <code>text-center</code> , <code>shadow-sm</code> |
| Components | gotowe elementy: karty, alerty, navbar, modal, dropdown                                                        |
| Forms      | pola formularzy, walidacja, checkboxy, selecty, input group                                                    |
| Helpers    | klasy pomocnicze, np. <code>clearfix</code> , <code>ratio</code> , <code>visually-hidden</code>                |

Bootstrap najlepiej sprawdza się, gdy:

- chcesz szybko zbudować estetyczny interfejs
- tworzysz panel administracyjny, formularze, dashboard, katalog produktów
- potrzebujesz responsywnej siatki bez pisania dużej ilości CSS
- zależy Ci na spójnych odstępach, kolorach i komponentach

Bootstrap nie zawsze jest najlepszym wyborem, gdy:

- projekt ma bardzo niestandardowy, artystyczny wygląd
- każda sekcja strony ma zupełnie inny system wizualny
- chcesz pisać własny design system od zera

Najważniejsza zasada: **Bootstrap przyspiesza pracę, ale nie zwalnia z myślenia o semantyce, dostępności i strukturze komponentów.**

## 1.2. Instalacja i konfiguracja Bootstrapa

W projekcie React najwygodniej instalować Bootstrapa jako paczkę npm:

```
npm install bootstrap
```

Następnie importujemy style w głównym pliku aplikacji. W Create React App będzie to zwykle `src/index.js`, a w Vite często `src/main.jsx`.

```
// Plik: src/index.js
import React from "react";
import ReactDOM from "react-dom/client";

import "bootstrap/dist/css/bootstrap.css";
import "./index.css";

import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Kolejność importów ma znaczenie:

```
import "bootstrap/dist/css/bootstrap.css";
import "./index.css";
```

Najpierw importujemy Bootstrapa, a dopiero potem własny CSS. Dzięki temu własne style mogą nadpisać klasy Bootstrapa, jeśli będzie to potrzebne.

Jeśli chcesz używać komponentów wymagających JavaScriptu, np. `modal`, `dropdown`, `collapse`, `offcanvas`, możesz dodatkowo zaimportować bundle JS:

```
import "bootstrap/dist/js/bootstrap.bundle.min.js";
```

W prostych projektach edukacyjnych najczęściej wystarczy sam CSS. JavaScript Bootstrapa dodajemy dopiero wtedy, gdy naprawdę korzystamy z komponentów interaktywnych opartych o atrybuty `data-bs-*`.

### Ikony Bootstrap Icons

Bootstrap nie zawiera ikon w podstawowej paczce. Ikony są osobną biblioteką:

```
npm install bootstrap-icons
```

```
import "bootstrap-icons/font/bootstrap-icons.css";
```

Przykład użycia:

```
<button className="btn btn-primary">  
  <i className="bi bi-save me-2"></i>  
  Zapisz  
</button>
```

### 1.3. Czysty Bootstrap CSS vs React-Bootstrap

W React są dwa popularne sposoby używania Bootstrapa.

#### 1. Klasy CSS Bootstrapa

To podejście stosowane w tej dokumentacji:

```
<button className="btn btn-primary">Zapisz</button>
```

Zalety:

- uczysz się prawdziwych klas Bootstrapa
- kod działa podobnie jak w zwykłym HTML
- nie trzeba instalować dodatkowej biblioteki komponentów
- łatwo mieszać Bootstrap z własnymi komponentami React

Wady:

- przy komponentach takich jak modal albo dropdown trzeba uważać na JS Bootstrapa
- długie `className` mogą być mniej czytelne

#### 2. React-Bootstrap

To osobna biblioteka komponentów:

```
npm install react-bootstrap bootstrap
```

Przykład:

```
import Button from "react-bootstrap/Button";  
  
function App() {  
  return <Button variant="primary">Zapisz</Button>;  
}
```

Zalety:

- komponenty są bardziej "reactowe"

- modale, dropdowny i zakładki łatwiej kontrolować stanem

Wady:

- uczysz się dodatkowego API
- dokumentacja React-Bootstrap różni się od dokumentacji Bootstrapa
- dla prostych projektów to często niepotrzebna warstwa

W tej dokumentacji używamy **klas CSS Bootstrapa**, bo są najbardziej uniwersalne.

#### 1.4. Kontenery i podstawowy układ strony

Kontener ogranicza szerokość treści i nadaje stronie czytelny układ. Bez kontenera elementy często przyklejają się do krawędzi ekranu.

Najważniejsze klasy:

| KLASA                        | DZIAŁANIE                                                           |
|------------------------------|---------------------------------------------------------------------|
| <code>container</code>       | responsywny kontener o maksymalnej szerokości                       |
| <code>container-fluid</code> | kontener na pełną szerokość ekranu                                  |
| <code>container-md</code>    | pełna szerokość do breakpointu <code>md</code> , potem ograniczenie |

```
function App() {  
  return (  
    <main className="container py-4">  
      <h1>Panel kursanta</h1>  
      <p className="lead">Treść jest czytelnie odsunięta od krawędzi.</p>  
    </main>  
  );  
}
```

Przykład pełnej szerokości:

```
<header className="container-fluid bg-dark text-white py-4">  
  <div className="container">  
    <h1 className="mb-0">Nagłówek strony</h1>  
  </div>  
</header>
```

Dobry wzorzec strony:

```
<div className="min-vh-100 bg-light">  
  <header className="bg-white border-bottom">  
    <div className="container py-3">Logo i nawigacja</div>  
  </header>  
  
  <main className="container py-4">  
    Treść strony  
  </main>  
  
  <footer className="border-top">  
    <div className="container py-3 text-muted">Stopka</div>  
  </footer>  
</div>
```

```
</footer>
</div>
```

## 1.5. System Grid — siatka 12-kolumnowa

Grid Bootstrapa opiera się na trzech elementach:

- `container` — ogranicza szerokość strony
- `row` — tworzy wiersz
- `col` / `col-*` — tworzy kolumny

Bootstrap dzieli wiersz na 12 części. Jeśli dasz `col-md-6`, element zajmie 6 z 12 kolumn, czyli połowę szerokości od breakpointu `md`.

Breakpointy:

| BREAKPOINT       | OD SZEROKOŚCI | PRZYKŁAD KLASY         |
|------------------|---------------|------------------------|
| <code>xs</code>  | domyślnie     | <code>col-12</code>    |
| <code>sm</code>  | 576px         | <code>col-sm-6</code>  |
| <code>md</code>  | 768px         | <code>col-md-4</code>  |
| <code>lg</code>  | 992px         | <code>col-lg-3</code>  |
| <code>xl</code>  | 1200px        | <code>col-xl-2</code>  |
| <code>xxl</code> | 1400px        | <code>col-xxl-2</code> |

Przykład: jedna kolumna na telefonie, trzy na komputerze.

```
<div className="container">
  <div className="row g-3">
    <div className="col-12 col-md-4">
      <div className="p-3 bg-primary text-white rounded">Kolumna 1</div>
    </div>
    <div className="col-12 col-md-4">
      <div className="p-3 bg-success text-white rounded">Kolumna 2</div>
    </div>
    <div className="col-12 col-md-4">
      <div className="p-3 bg-danger text-white rounded">Kolumna 3</div>
    </div>
  </div>
</div>
```

`g-3` oznacza odstęp między kolumnami i wierszami. Można rozdzielić odstępy:

| KLASA             | DZIAŁANIE                   |
|-------------------|-----------------------------|
| <code>g-0</code>  | brak odstępow               |
| <code>g-3</code>  | odstępy w pionie i poziomie |
| <code>gx-4</code> | odstępy tylko poziome       |
| <code>gy-2</code> | odstępy tylko pionowe       |

Układ panelowy: sidebar + treść główna.

```

<div className="container py-4">
  <div className="row g-4">
    <aside className="col-12 col-lg-3">
      <div className="list-group">
        <button className="list-group-item list-group-item-action active">Profil</button>
        <button className="list-group-item list-group-item-action">Kursy</button>
        <button className="list-group-item list-group-item-action">Ustawienia</button>
      </div>
    </aside>

    <section className="col-12 col-lg-9">
      <div className="card">
        <div className="card-body">
          <h2 className="h4">Treść główna</h2>
          <p>Na telefonie sidebar będzie nad treścią, a na dużym ekranie obok.</p>
        </div>
      </div>
    </section>
  </div>
</div>

```

Szybkie automatyczne kolumny:

```

<div className="row row-cols-1 row-cols-md-2 row-cols-xl-4 g-3">
  {produkty.map((produkt) => (
    <div className="col" key={produkt.id}>
      <div className="card h-100">
        <div className="card-body">{produkt.nazwa}</div>
      </div>
    </div>
  ))}
</div>

```

`row-cols-*` jest bardzo wygodne przy kartach, kafelkach i galeriach.

## 1.6. Flexbox i szybkie wyrównywanie elementów

Grid służy do większych układów strony. Flexbox przydaje się do układania elementów wewnątrz komponentu: przycisków, nagłówków kart, pasków narzędzi, ikon i opisów.

Najważniejsze klasy:

| KLASA                                | DZIAŁANIE                                          |
|--------------------------------------|----------------------------------------------------|
| <code>d-flex</code>                  | włącza flexbox                                     |
| <code>flex-column</code>             | układa elementy pionowo                            |
| <code>justify-content-between</code> | rozsuwa elementy do boków                          |
| <code>justify-content-center</code>  | centruje w osi głównej                             |
| <code>align-items-center</code>      | wyrównuje w osi poprzecznej                        |
| <code>gap-2</code>                   | dodaje odstęp między elementami                    |
| <code>flex-wrap</code>               | pozwała elementom zawijać się do kolejnego wiersza |

```
<div className="d-flex justify-content-between align-items-center p-3 border rounded">
  <div>
    <h2 className="h5 mb-0">Lista zadań</h2>
    <small className="text-muted">3 zadania do wykonania</small>
  </div>

  <div className="d-flex gap-2">
    <button className="btn btn-outline-secondary btn-sm">Filtruj</button>
    <button className="btn btn-primary btn-sm">Dodaj</button>
  </div>
</div>
```

Przykład z pionowym układem i przyciskiem na dole:

```
<div className="card h-100">
  <div className="card-body d-flex flex-column">
    <h3 className="h5">Kurs React</h3>
    <p className="text-muted">Opis może mieć różną długość.</p>

    <button className="btn btn-primary mt-auto">
      Zobacz kurs
    </button>
  </div>
</div>
```

`mt-auto` wypycha przycisk na dół karty, jeśli rodzic ma `d-flex flex-column`.

## 1.7. Display, widoczność, pozycjonowanie i overflow

Bootstrap ma klasy do szybkiej kontroli sposobu wyświetlania elementów.

| KLASA                       | DZIAŁANIE       |
|-----------------------------|-----------------|
| <code>d-none</code>         | ukrywa element  |
| <code>d-block</code>        | element blokowy |
| <code>d-inline</code>       | element liniowy |
| <code>d-inline-block</code> | liniowy blok    |
| <code>d-flex</code>         | flexbox         |
| <code>d-grid</code>         | CSS grid        |

Klasy mogą być responsywne:

```
<div className="d-none d-md-block">
  Ten panel widać od tabletów w górę.
</div>

<div className="d-block d-md-none">
  Ten panel widać tylko na telefonach.
</div>
```

Pozycjonowanie:

```
<div className="position-relative border rounded p-4">
  <span className="position-absolute top-0 start-100 translate-middle badge rounded-pill bg-danger">
    3
  </span>
  Powiadomienia
</div>
```

Przydatne klasy:

| KLASA                                                                                  | DZIAŁANIE                                      |
|----------------------------------------------------------------------------------------|------------------------------------------------|
| <code>position-relative</code>                                                         | element staje się punktem odniesienia          |
| <code>position-absolute</code>                                                         | pozycjonowanie względem rodzica                |
| <code>top-0</code> , <code>bottom-0</code> , <code>start-0</code> , <code>end-0</code> | przyklejenie do krawędzi                       |
| <code>translate-middle</code>                                                          | przesunięcie o połowę własnego rozmiaru        |
| <code>sticky-top</code>                                                                | element zostaje przy górze podczas przewijania |
| <code>overflow-auto</code>                                                             | przewijanie, gdy treść się nie mieści          |
| <code>overflow-hidden</code>                                                           | ukrycie wystającej treści                      |

Przykład przewijanej listy:

```
<div className="border rounded overflow-auto" style={{ maxHeight: "240px" }}>
  {wiadomosci.map((msg) => (
    <div className="border-bottom p-2" key={msg.id}>
      {msg.tresc}
    </div>
  ))}
</div>
```



```
    </div>
  )})
</div>
```

## 1.8. Spacing, wymiary, obramowania i cienie

Spacing to system marginesów i paddingów. Format klasy:

[właściwość][strona]-[rozmiar]

Właściwości:

- **m** — margin
- **p** — padding

Strony:

- **t** — top
- **b** — bottom
- **s** — start, czyli lewa strona w językach LTR
- **e** — end, czyli prawa strona w językach LTR
- **x** — lewo i prawo
- **y** — góra i dół
- brak strony — wszystkie strony

Rozmiary:

| KLASA       | ZNACZENIE             |
|-------------|-----------------------|
| <b>0</b>    | brak odstępu          |
| <b>1</b>    | mały odstęp           |
| <b>2</b>    | trochę większy        |
| <b>3</b>    | standardowy odstęp    |
| <b>4</b>    | duży odstęp           |
| <b>5</b>    | bardzo duży odstęp    |
| <b>auto</b> | automatyczny margines |

```
<section className="container py-5">
  <div className="mx-auto p-4 border rounded shadow-sm" style={{ maxWidth: "520px" }}>
    <h2 className="mb-3">Logowanie</h2>
    <p className="text-muted mb-4">Wpisz dane dostępowe do konta.</p>
  </div>
</section>
```

Wymiary:

| KLASA                   | DZIAŁANIE                     |
|-------------------------|-------------------------------|
| w-25, w-50, w-75, w-100 | szerokość procentowa          |
| h-25, h-50, h-75, h-100 | wysokość procentowa           |
| mw-100                  | maksymalna szerokość 100%     |
| min-vh-100              | minimum pełna wysokość ekranu |

Obramowania i zaokrąglenia:

```
<div className="border border-primary rounded-3 p-3">
  Ramka primary i zaokrąglone rogi
</div>


```

Cienie:

| KLASA       | EFEKT            |
|-------------|------------------|
| shadow-none | brak cienia      |
| shadow-sm   | mały cień        |
| shadow      | standardowy cień |
| shadow-lg   | duży cień        |

1.9. Typografia, kolory, tła i tryb ciemny

Bootstrap dostarcza gotowe klasy typograficzne.

| KLASA                             | DZIAŁANIE                            |
|-----------------------------------|--------------------------------------|
| display-1 ... display-6           | bardzo duże nagłówki                 |
| h1 ... h6                         | wygląd nagłówka bez zmiany znacznika |
| lead                              | większy akapit wprowadzający         |
| small                             | mniejszy tekst                       |
| fw-bold                           | pogrubienie                          |
| fw-normal                         | normalna grubość                     |
| fst-italic                        | kursywa                              |
| text-start, text-center, text-end | wyrównanie tekstu                    |
| text-uppercase                    | wielkie litery                       |
| text-truncate                     | ucięcie tekstu z wielokropkiem       |

```
<section className="bg-light p-5 text-center">
  <h1 className="display-5 fw-bold">Kurs React i Bootstrap</h1>
  <p className="lead text-muted mb-0">
    Szybkie budowanie czytelnych interfejsów.
  </p>
</section>
```

```
</p>
</section>
```

Kolory semantyczne:

| NAZWA                  | TYPOWE ZNACZENIE  |
|------------------------|-------------------|
| <code>primary</code>   | główna akcja      |
| <code>secondary</code> | akcja drugorzędna |
| <code>success</code>   | sukces            |
| <code>danger</code>    | błąd, usuwanie    |
| <code>warning</code>   | ostrzeżenie       |
| <code>info</code>      | informacja        |
| <code>light</code>     | jasne tło         |
| <code>dark</code>      | ciemne tło        |

```
<div className="p-3 bg-success-subtle text-success-emphasis border border-success rounded">
  Operacja zakończona powodzeniem.
</div>
```

W Bootstrapie 5.3 dostępne są też klasy typu `bg-primary-subtle`, `text-primary-emphasis`, `border-primary-subtle`. Są wygodne, gdy pełne `bg-primary text-white` byłoby zbyt mocne.

Tryb ciemny można aktywować atrybutem `data-bs-theme`.

```
function App() {
  const dark = true;

  return (
    <div data-bs-theme={dark ? "dark" : "light"} className="min-vh-100 p-4">
      <div className="card">
        <div className="card-body">
          <h1 className="h4">Karta dopasowana do motywu</h1>
          <button className="btn btn-primary">Akcja</button>
        </div>
      </div>
    </div>
  );
}
```

## 1.10. Przyciski, grupy przycisków i stany

Każdy przycisk Bootstrapa zaczyna się od klasy `btn`.

```
<button className="btn btn-primary">Zapisz</button>
<button className="btn btn-outline-secondary">Anuluj</button>
<button className="btn btn-danger">Usuń</button>
```

Najczęstsze warianty:

| KLASA                                     | ZASTOSOWANIE            |
|-------------------------------------------|-------------------------|
| <code>btn-primary</code>                  | główna akcja            |
| <code>btn-secondary</code>                | akcja pomocnicza        |
| <code>btn-success</code>                  | potwierdzenie           |
| <code>btn-danger</code>                   | usuwanie lub błąd       |
| <code>btn-warning</code>                  | ostrzeżenie             |
| <code>btn-outline-*</code>                | przycisk z obramowaniem |
| <code>btn-sm</code> , <code>btn-lg</code> | rozmiar                 |

Przyciski w React często zależą od stanu:

```
function ZapiszButton({ zapisuje, poprawny }) {
  return (
    <button className="btn btn-primary" disabled={zapisuje || !poprawny}>
      {zapisuje ? "Zapisywanie..." : "Zapisz"}
    </button>
  );
}
```

Grupa przycisków:

```
<div className="btn-group" role="group" aria-label="Widok danych">
  <button type="button" className="btn btn-outline-primary active">Karty</button>
  <button type="button" className="btn btn-outline-primary">Tabela</button>
  <button type="button" className="btn btn-outline-primary">Wykres</button>
</div>
```

Pełna szerokość i układ pionowy:

```
<div className="d-grid gap-2">
  <button className="btn btn-primary">Zapisz</button>
  <button className="btn btn-outline-secondary">Wróć</button>
</div>
```

W formularzach zawsze ustawiaj `type`:

```
<button type="submit" className="btn btn-primary">Wyślij</button>
<button type="button" className="btn btn-outline-secondary">Anuluj</button>
```

Bez `type="button"` przycisk wewnątrz formularza domyślnie zachowuje się jak submit.

### 1.11. Formularze — pola, selecty, checkboxy i input group

Podstawowe klasy formularzy:

| ELEMENT                                    | KLASA                         |
|--------------------------------------------|-------------------------------|
| <code>input</code> , <code>textarea</code> | <code>form-control</code>     |
| <code>select</code>                        | <code>form-select</code>      |
| <code>label</code>                         | <code>form-label</code>       |
| checkbox/radio wrapper                     | <code>form-check</code>       |
| checkbox/radio input                       | <code>form-check-input</code> |
| checkbox/radio label                       | <code>form-check-label</code> |
| tekst pomocniczy                           | <code>form-text</code>        |

```
function FormularzKontaktowy() {
  return (
    <form className="border rounded p-4 bg-light">
      <div className="mb-3">
        <label htmlFor="email" className="form-label">Adres e-mail</label>
        <input
          id="email"
          type="email"
          className="form-control"
          placeholder="jan@example.com"
        />
        <div className="form-text">Nie udostępniamy adresu innym osobom.</div>
      </div>

      <div className="mb-3">
        <label htmlFor="temat" className="form-label">Temat</label>
        <select id="temat" className="form-select">
          <option value="">Wybierz temat</option>
          <option value="konto">Konto</option>
          <option value="platnosci">Płatności</option>
          <option value="inne">Inne</option>
        </select>
      </div>

      <div className="mb-3">
        <label htmlFor="tresc" className="form-label">Treść</label>
        <textarea id="tresc" className="form-control" rows="4"></textarea>
      </div>

      <button type="submit" className="btn btn-primary">Wyślij</button>
    </form>
  );
}
```

Checkbox i switch:

```
<div className="form-check mb-2">
  <input className="form-check-input" type="checkbox" id="newsletter" />
```

```

<label className="form-check-label" htmlFor="newsletter">
  Chcę otrzymywać newsletter
</label>
</div>

<div className="form-check form-switch">
  <input className="form-check-input" type="checkbox" role="switch" id="tryb" />
  <label className="form-check-label" htmlFor="tryb">
    Tryb ciemny
  </label>
</div>

```

Radio:

```

<div className="form-check">
  <input className="form-check-input" type="radio" name="plan" id="basic" />
  <label className="form-check-label" htmlFor="basic">Basic</label>
</div>
<div className="form-check">
  <input className="form-check-input" type="radio" name="plan" id="pro" />
  <label className="form-check-label" htmlFor="pro">Pro</label>
</div>

```

Input group, czyli pole z dodatkiem:

```

<div className="input-group mb-3">
  <span className="input-group-text">PLN</span>
  <input type="number" className="form-control" placeholder="Cena" />
  <button className="btn btn-outline-secondary" type="button">
    Przelicz
  </button>
</div>

```

Formularz kontrolowany w React:

```

import { useState } from "react";

function Formularz() {
  const [email, setEmail] = useState("");

  return (
    <form>
      <label htmlFor="email" className="form-label">E-mail</label>
      <input
        id="email"
        className="form-control"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />
    </form>
  );
}

```

```
);  
}
```

### 1.12. Walidacja formularzy i floating labels

Bootstrap daje klasy wizualne do walidacji, ale sama logika walidacji należy do Reacta.

| KLASA                         | EFEKT             |
|-------------------------------|-------------------|
| <code>is-valid</code>         | zielone pole      |
| <code>is-invalid</code>       | czerwone pole     |
| <code>valid-feedback</code>   | komunikat sukcesu |
| <code>invalid-feedback</code> | komunikat błędu   |

```
import { useState } from "react";  
  
function WalidowanyEmail() {  
  const [email, setEmail] = useState("");  
  const dotkniete = email.length > 0;  
  const poprawny = email.includes("@") && email.includes(".");  
  
  let klasa = "form-control";  
  if (dotkniete && poprawny) klasa += " is-valid";  
  if (dotkniete && !poprawny) klasa += " is-invalid";  
  
  return (  
    <div className="mb-3">  
      <label htmlFor="email" className="form-label">E-mail</label>  
      <input  
        id="email"  
        className={klasa}  
        value={email}  
        onChange={(e) => setEmail(e.target.value)}  
      />  
      <div className="valid-feedback">Adres wygląda poprawnie.</div>  
      <div className="invalid-feedback">Podaj poprawny adres e-mail.</div>  
    </div>  
  );  
}
```

Floating labels:

```
<div className="form-floating mb-3">  
  <input  
    type="email"  
    className="form-control"  
    id="floatingEmail"  
    placeholder="jan@example.com"  
  />  
</div>
```

```
<label htmlFor="floatingEmail">Adres e-mail</label>
</div>
```

Przy `form-floating` placeholder nadal jest potrzebny technicznie, nawet jeśli użytkownik widzi label.

Walidacja całego formularza:

```
function Rejestracja() {
  const [imie, setImie] = useState("");
  const [zgoda, setZgoda] = useState(false);

  const poprawneImie = imie.trim().length ≥ 3;
  const moznaWyslac = poprawneImie && zgoda;

  function handleSubmit(e) {
    e.preventDefault();
    if (!moznaWyslac) return;
    alert("Formularz wysłany");
  }

  return (
    <form className="p-4 border rounded" onSubmit={handleSubmit}>
      <div className="mb-3">
        <label htmlFor="imie" className="form-label">Imię</label>
        <input
          id="imie"
          className={`form-control ${imie && !poprawneImie ? "is-invalid" : ""}`}
          value={imie}
          onChange={(e) => setImie(e.target.value)}
        />
        <div className="invalid-feedback">Minimum 3 znaki.</div>
      </div>

      <div className="form-check mb-3">
        <input
          id="zgoda"
          className="form-check-input"
          type="checkbox"
          checked={zgoda}
          onChange={(e) => setZgoda(e.target.checked)}
        />
        <label className="form-check-label" htmlFor="zgoda">
          Akceptuję regulamin
        </label>
      </div>

      <button className="btn btn-primary" disabled={!moznaWyslac}>
        Zarejestruj
      </button>
    </form>
  );
}
```



### 1.13. Nawigacja — navbar, nav, tabs i breadcrumbs

Navbar to górny pasek nawigacyjny. W najprostszej wersji nie musi mieć JavaScriptu.

```
<nav className="navbar bg-dark navbar-dark">
  <div className="container">
    <a className="navbar-brand" href="/">Moja aplikacja</a>
    <div className="d-flex gap-2">
      <a className="btn btn-outline-light btn-sm" href="/login">Logowanie</a>
      <a className="btn btn-primary btn-sm" href="/register">Rejestracja</a>
    </div>
  </div>
</nav>
```

Responsywny navbar z collapse wymaga JS Bootstrapa:

```
<nav className="navbar navbar-expand-lg bg-body-tertiary border-bottom">
  <div className="container">
    <a className="navbar-brand fw-bold" href="/">Kursy</a>

    <button
      className="navbar-toggler"
      type="button"
      data-bs-toggle="collapse"
      data-bs-target="#mainNav"
      aria-controls="mainNav"
      aria-expanded="false"
      aria-label="Przełącz nawigację"
    >
      <span className="navbar-toggler-icon"></span>
    </button>

    <div className="collapse navbar-collapse" id="mainNav">
      <ul className="navbar-nav ms-auto">
        <li className="nav-item"><a className="nav-link active" href="/">Start</a></li>
        <li className="nav-item"><a className="nav-link" href="/kursy">Kursy</a></li>
        <li className="nav-item"><a className="nav-link" href="/kontakt">Kontakt</a></li>
      </ul>
    </div>
  </div>
</nav>
```

**nav** i zakładki sterowane stanem React:

```
import { useState } from "react";

function Zakladki() {
  const [aktywny, setAktywny] = useState("opis");

  return (
    <
      <ul className="nav nav-tabs mb-3">
```

```

    <li className="nav-item">
      <button className={`nav-link ${aktywny === "opis" ? "active" : ""}`} onClick={() => setAktywny("opis")}>
        Opis
      </button>
    </li>
    <li className="nav-item">
      <button className={`nav-link ${aktywny === "opinie" ? "active" : ""}`} onClick={() => setAktywny("opinie")}>
        Opinie
      </button>
    </li>
  </ul>

  {aktywny === "opis" && <p>Opis produktu...</p>}
  {aktywny === "opinie" && <p>Lista opinii...</p>}
</>
);
}

```

Breadcrumbs:

```

<nav aria-label="breadcrumb">
  <ol className="breadcrumb">
    <li className="breadcrumb-item"><a href="/">Start</a></li>
    <li className="breadcrumb-item"><a href="/kursy">Kursy</a></li>
    <li className="breadcrumb-item active" aria-current="page">React</li>
  </ol>
</nav>

```

#### 1.14. Karty, list group, badge i układy kafelkowe

Karta składa się zwykle z `.card`, `.card-body`, opcjonalnie `.card-header`, `.card-footer`, `.card-title`, `.card-text`.

```

<div className="card shadow-sm">
  <div className="card-header bg-white">
    Polecany kurs
  </div>
  <div className="card-body">
    <h3 className="card-title h5">React od podstaw</h3>
    <p className="card-text text-muted">
      Komponenty, stan, formularze i praktyczne projekty.
    </p>
    <a href="/kurs/react" className="btn btn-primary">Zobacz</a>
  </div>
  <div className="card-footer text-muted">
    12 lekcji
  </div>
</div>

```

Karty w siatce:

```

<div className="row row-cols-1 row-cols-md-2 row-cols-xl-3 g-4">
  {kursy.map((kurs) => (
    <div className="col" key={kurs.id}>
      <div className="card h-100 shadow-sm">
        <div className="card-body d-flex flex-column">
          <div className="d-flex justify-content-between align-items-start">
            <h3 className="h5 card-title">{kurs.nazwa}</h3>
            <span className="badge bg-primary">{kurs.poziom}</span>
          </div>

          <p className="card-text text-muted">{kurs.opis}</p>

          <div className="mt-auto d-flex justify-content-between align-items-center">
            <strong>{kurs.cena} zł</strong>
            <button className="btn btn-outline-primary btn-sm">Szczegóły</button>
          </div>
        </div>
      </div>
    </div>
  ))}
</div>

```

List group:

```

<div className="list-group">
  <button className="list-group-item list-group-item-action active">
    Konto
  </button>
  <button className="list-group-item list-group-item-action">
    Bezpieczeństwo
  </button>
  <button className="list-group-item list-group-item-action d-flex justify-content-between align-items-center">
    Powiadomienia
    <span className="badge bg-danger rounded-pill">4</span>
  </button>
</div>

```

Badge:

```

<h2>
  Zamówienia <span className="badge bg-secondary">12</span>
</h2>

<span className="badge rounded-pill text-bg-success">Aktywne</span>
<span className="badge rounded-pill text-bg-warning">Oczekuje</span>
<span className="badge rounded-pill text-bg-danger">Błąd</span>

```

## 1.15. Tabele, paginacja i prezentacja danych

Tabele są dobre dla danych porównywalnych: użytkowników, zamówień, ocen, historii operacji.

```
<div className="table-responsive">
  <table className="table table-striped table-hover align-middle">
    <thead className="table-light">
      <tr>
        <th>ID</th>
        <th>Klient</th>
        <th>Status</th>
        <th className="text-end">Kwota</th>
        <th className="text-end">Akcje</th>
      </tr>
    </thead>
    <tbody>
      {zamowienia.map((z) => (
        <tr key={z.id}>
          <td>#{z.id}</td>
          <td>{z.klient}</td>
          <td><span className="badge text-bg-success">{z.status}</span></td>
          <td className="text-end">{z.kwota} zł</td>
          <td className="text-end">
            <button className="btn btn-sm btn-outline-primary">Podgląd</button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
```

Najczęstsze klasy tabel:

| KLASA                         | EFEKT                          |
|-------------------------------|--------------------------------|
| <code>table</code>            | bazowy styl tabeli             |
| <code>table-striped</code>    | pasy w wierszach               |
| <code>table-hover</code>      | podświetlenie po najechaniu    |
| <code>table-bordered</code>   | obramowania komórek            |
| <code>table-sm</code>         | ciaśniejsza tabela             |
| <code>align-middle</code>     | pionowe wyśrodkowanie          |
| <code>table-responsive</code> | przewijanie na małych ekranach |

Paginacja:

```
<nav aria-label="Strony wyników">
  <ul className="pagination justify-content-center">
    <li className="page-item disabled">
      <button className="page-link">Poprzednia</button>
    </li>
    <li className="page-item active">
      <button className="page-link">1</button>
    </li>
```

```

    <li className="page-item">
      <button className="page-link">2</button>
    </li>
    <li className="page-item">
      <button className="page-link">Następna</button>
    </li>
  </ul>
</nav>

```

W React paginacja zwykle wynika ze stanu:

```

const [strona, setStrona] = useState(1);
const naStrone = 10;
const start = (strona - 1) * naStrone;
const widoczne = dane.slice(start, start + naStrone);

```

## 1.16. Alerty, spinnery, progress, placeholdery i toast

Alert:

```

{blad && (
  <div className="alert alert-danger" role="alert">
    {blad}
  </div>
)}

{sukces && (
  <div className="alert alert-success" role="alert">
    Dane zostały zapisane.
  </div>
)}

```

Spinner:

```

{ladowanie && (
  <div className="d-flex justify-content-center py-5">
    <div className="spinner-border text-primary" role="status">
      <span className="visually-hidden">Ładowanie... </span>
    </div>
  </div>
)}

```

Progress:

```

<div className="progress" role="progressbar" aria-label="Postęp" aria-valuenow={75} aria-valuemin="0" aria-valuemax="100">
  <div className="progress-bar progress-bar-striped bg-success" style={{ width: "75%" }}>
    75%
  </div>
</div>

```

Placeholder, czyli szkielet ładowania:

```

<div className="card" aria-hidden="true">
  <div className="card-body">
    <h5 className="card-title placeholder-glow">
      <span className="placeholder col-6"></span>
    </h5>
    <p className="card-text placeholder-glow">
      <span className="placeholder col-7"></span>
      <span className="placeholder col-4"></span>
      <span className="placeholder col-4"></span>
    </p>
  </div>
</div>

```

Toast w Bootstrapie wymaga JavaScriptu albo własnej kontroli Reactem. Prosty toast kontrolowany stanem:

```

{pokazToast && (
  <div className="position-fixed bottom-0 end-0 p-3" style={{ zIndex: 10 }}>
    <div className="toast show">
      <div className="toast-header">
        <strong className="me-auto">System</strong>
        <button type="button" className="btn-close" onClick={() => setPokazToast(false)}></button>
      </div>
      <div className="toast-body">
        Zapisano zmiany.
      </div>
    </div>
  </div>
)}

```

### 1.17. Komponenty wymagające JavaScriptu

Część komponentów Bootstrapa działa wyłącznie na CSS, np. przyciski, karty, alerty, formularze, badge, tabele. Inne wymagają JavaScriptu Bootstrapa:

- modal
- dropdown
- collapse
- offcanvas
- tooltip
- popover
- carousel
- toast, jeśli używasz API Bootstrapa

Jeśli używasz atrybutów `data-bs-*`, dodaj:

```
import "bootstrap/dist/js/bootstrap.bundle.min.js";
```

## Collapse / Accordion

```
<div className="accordion" id="faq">
  <div className="accordion-item">
    <h2 className="accordion-header">
      <button
        className="accordion-button"
        type="button"
        data-bs-toggle="collapse"
        data-bs-target="#odp1"
      >
        Czym jest Bootstrap?
      </button>
    </h2>
    <div id="odp1" className="accordion-collapse collapse show" data-bs-parent="#faq">
      <div className="accordion-body">
        To framework CSS z gotowymi klasami i komponentami.
      </div>
    </div>
  </div>
</div>
```

## Modal

```
<button type="button" className="btn btn-primary" data-bs-toggle="modal" data-bs-target="#potwierdzModal">
  Usuń konto
</button>

<div className="modal fade" id="potwierdzModal" tabIndex="-1" aria-labelledby="potwierdzLabel" aria-hidden="true">
  <div className="modal-dialog">
    <div className="modal-content">
      <div className="modal-header">
        <h1 className="modal-title fs-5" id="potwierdzLabel">Potwierdzenie</h1>
        <button type="button" className="btn-close" data-bs-dismiss="modal" aria-label="Zamknij"></button>
      </div>
      <div className="modal-body">
        Czy na pewno chcesz usunąć konto?
      </div>
      <div className="modal-footer">
        <button type="button" className="btn btn-secondary" data-bs-dismiss="modal">Anuluj</button>
        <button type="button" className="btn btn-danger">Usuń</button>
      </div>
    </div>
  </div>
</div>
```

## Offcanvas

```
<button className="btn btn-outline-primary" type="button" data-bs-toggle="offcanvas" data-bs-target="#menuBoczne">
  Menu
</button>

<div className="offcanvas offcanvas-start" tabIndex="-1" id="menuBoczne">
  <div className="offcanvas-header">
    <h5 className="offcanvas-title">Nawigacja</h5>
    <button type="button" className="btn-close" data-bs-dismiss="offcanvas" aria-label="Zamknij"></button>
  </div>
  <div className="offcanvas-body">
    <div className="list-group">
      <a href="/" className="list-group-item list-group-item-action">Start</a>
      <a href="/konto" className="list-group-item list-group-item-action">Konto</a>
    </div>
  </div>
</div>
```

## Dropdown

```
<div className="dropdown">
  <button className="btn btn-secondary dropdown-toggle" type="button" data-bs-toggle="dropdown" aria-expanded="false">
    Akcje
  </button>
  <ul className="dropdown-menu">
    <li><button className="dropdown-item" type="button">Edytuj</button></li>
    <li><button className="dropdown-item" type="button">Duplikuj</button></li>
    <li><hr className="dropdown-divider" /></li>
    <li><button className="dropdown-item text-danger" type="button">Usuń</button></li>
  </ul>
</div>
```

W większej aplikacji React często lepiej kontrolować modal, zakładki albo toast stanem Reacta niż mieszać logikę z `data-bs-*`. Dla prostych projektów szkolnych atrybuty Bootstrapa są jednak wystarczające.

### 1.18. Dostępność i semantyka w Bootstrapie

Bootstrap daje dobre style, ale dostępność nadal zależy od kodu HTML.

Najważniejsze zasady:

- używaj prawdziwych znaczników: `button` do akcji, `a` do linków
- każdy `input` powinien mieć `label`
- nie usuwaj widocznego fokusu bez zapewnienia alternatywy
- przy spinnerach dodawaj tekst dla czytników ekranu: `visually-hidden`
- przy modalach, dropdownach i navbarach zachowuj atrybuty `aria-*`
- nie przekazuj znaczenia tylko kolorem

Przykład dobrego pola:

```
<label htmlFor="haslo" className="form-label">Hasło</label>

```



```

    id="haslo"
    type="password"
    className="form-control"
    aria-describedby="hasloPomoc"
  />
  <div id="hasloPomoc" className="form-text">
    Hasło powinno mieć minimum 8 znaków.
  </div>

```

Klasa `visually-hidden` ukrywa tekst wizualnie, ale zostawia go dla czytników ekranu:

```

<button className="btn btn-outline-danger">
  <span aria-hidden="true">×</span>
  <span className="visually-hidden">Usuń element</span>
</button>

```

### 1.19. Nadpisywanie Bootstrapa i własny motyw

Najprostszy sposób nadpisywania Bootstrapa to własny plik CSS importowany po Bootstrapie.

```

import "bootstrap/dist/css/bootstrap.css";
import "../index.css";

```

Przykład:

```

/* Plik: src/index.css */
.app-card {
  border-radius: 0.75rem;
}

.btn-primary {
  background-color: #0f766e;
  border-color: #0f766e;
}

.btn-primary:hover {
  background-color: #115e59;
  border-color: #115e59;
}

```

Nie nadpisuj wszystkiego globalnie bez potrzeby. Jeśli zmiana dotyczy jednego komponentu, lepiej dodać własną klasę:

```

<div className="card app-card shadow-sm">
  <div className="card-body">Treść</div>
</div>

```

Bootstrap 5 używa też zmiennych CSS. Można zmienić wygląd wybranego fragmentu:

```

<div
  className="card"
  style={{
    "--bs-card-border-color": "#0d6efd",
    "--bs-card-border-width": "2px",
  }}
>
  <div className="card-body">Karta z lokalnie zmienioną ramką</div>
</div>

```

Praktyczna zasada:

- klasy Bootstrapa stosuj do typowych rzeczy: spacing, grid, kolory, komponenty
- własne klasy stosuj do wyglądu specyficznego dla projektu
- unikaj bardzo długich `style={{ ... }}` dla zwykłego CSS

## 1.20. Złożony przykład praktyczny: Panel użytkownika

Poniższy przykład łączy siatkę, navbar, karty, tabele, formularz, alert, badge, progress, tryb ciemny i klasy responsywne.

```

// Plik: src/App.js
import { useState } from "react";
import "bootstrap/dist/css/bootstrap.css";

const zadaniaStart = [
  { id: 1, nazwa: "Uzupełnić profil", status: "Gotowe" },
  { id: 2, nazwa: "Dodać projekt", status: "W trakcie" },
  { id: 3, nazwa: "Wysłać formularz", status: "Oczekuje" },
];

function App() {
  const [trybCiemny, setTrybCiemny] = useState(false);
  const [zadania] = useState(zadaniaStart);
  const [email, setEmail] = useState("");
  const [komunikat, setKomunikat] = useState("");

  const ukonczone = zadania.filter((z) => z.status === "Gotowe").length;
  const postep = Math.round((ukonczone / zadania.length) * 100);

  function zapiszEmail(e) {
    e.preventDefault();
    if (!email.includes("@")) {
      setKomunikat("Podaj poprawny adres e-mail.");
      return;
    }
    setKomunikat("Adres zapisany poprawnie.");
  }

  return (
    <div data-bs-theme={trybCiemny ? "dark" : "light"} className="min-vh-100 bg-body-tertiary">
      <nav className="navbar navbar-expand bg-body border-bottom">
        <div className="container">
          <span className="navbar-brand fw-bold">Panel kursanta</span>

```

```

<div className="form-check form-switch ms-auto">
  <input
    id="darkMode"
    className="form-check-input"
    type="checkbox"
    checked={trybCiemny}
    onChange={(e) => setTrybCiemny(e.target.checked)}
  />
  <label className="form-check-label" htmlFor="darkMode">
    Tryb ciemny
  </label>
</div>
</div>
</nav>

<main className="container py-4">
  {komunikat && (
    <div className={`alert ${komunikat.includes("poprawnie")} ? "alert-success" : "alert-danger"}` role="alert">
      {komunikat}
    </div>
  )}

  <div className="row g-4">
    <aside className="col-12 col-lg-3">
      <div className="card shadow-sm">
        <div className="card-body text-center">
          <div className="rounded-circle bg-primary text-white d-flex align-items-center justify-content-center mx-auto mb-3"
            style={{ width: "72px", height: "72px" }}>
            JK
          </div>
          <h1 className="h5 mb-1">Jan Kowalski</h1>
          <p className="text-muted mb-3">Frontend Developer</p>
          <div className="d-grid gap-2">
            <button className="btn btn-primary btn-sm">Edytuj profil</button>
            <button className="btn btn-outline-secondary btn-sm">Ustawienia</button>
          </div>
        </div>
      </div>
    </div>

    <section className="col-12 col-lg-9">
      <div className="row g-3 mb-4">
        <div className="col-12 col-md-4">
          <div className="card h-100 shadow-sm">
            <div className="card-body">
              <p className="text-muted mb-1">Postęp</p>
              <h2 className="h3">{postep}%</h2>
              <div className="progress">
                <div className="progress-bar" style={{ width: `${postep}%` }}></div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </section>
  </div>

```

```

<div className="col-12 col-md-4">
  <div className="card h-100 shadow-sm">
    <div className="card-body">
      <p className="text-muted mb-1">Zadania</p>
      <h2 className="h3">{zadania.length}</h2>
      <span className="badge text-bg-info">Aktywne konto</span>
    </div>
  </div>
</div>

<div className="col-12 col-md-4">
  <div className="card h-100 shadow-sm">
    <div className="card-body">
      <p className="text-muted mb-1">Plan</p>
      <h2 className="h3">Pro</h2>
      <button className="btn btn-outline-primary btn-sm">Zmień plan</button>
    </div>
  </div>
</div>
</div>

<div className="card shadow-sm mb-4">
  <div className="card-header bg-body d-flex justify-content-between align-items-center">
    <h2 className="h5 mb-0">Lista zadań</h2>
    <span className="badge text-bg-secondary">{ukonczone}/{zadania.length}</span>
  </div>
  <div className="table-responsive">
    <table className="table table-hover align-middle mb-0">
      <thead>
        <tr>
          <th>Zadanie</th>
          <th>Status</th>
          <th className="text-end">Akcja</th>
        </tr>
      </thead>
      <tbody>
        {zadania.map((zadanie) => (
          <tr key={zadanie.id}>
            <td>{zadanie.nazwa}</td>
            <td>
              <span className={`badge ${
                zadanie.status === "Gotowe" ? "text-bg-success" :
                zadanie.status === "W trakcie" ? "text-bg-warning" :
                "text-bg-secondary"
              }`}>
                {zadanie.status}
              </span>
            </td>
            <td className="text-end">
              <button className="btn btn-sm btn-outline-primary">Szczegóły</button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
</div>

```

```

        </tbody>
      </table>
    </div>
  </div>

  <form className="card shadow-sm" onSubmit={zapiszEmail}>
    <div className="card-body">
      <h2 className="h5">Powiadomienia e-mail</h2>
      <div className="input-group">
        <span className="input-group-text">@</span>
        <input
          type="email"
          className="form-control"
          placeholder="jan@example.com"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <button className="btn btn-primary" type="submit">
          Zapisz
        </button>
      </div>
    </div>
  </form>
</section>
</div>
</main>
</div>

);
}

export default App;

```

## 2. Bootstrap bez Reacta: CDN i zwykły HTML

Bootstrap nie jest biblioteką Reacta. Można go używać w zwykłej stronie HTML, w aplikacji React, w widoku generowanym przez backend albo w dowolnym innym projekcie frontendowym. Najprostszy wariant to CDN, czyli dołączenie gotowych plików CSS i JavaScript przez linki.

### 2.1. Minimalny plik HTML z Bootstrapem

```

<!doctype html>
<html lang="pl">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Bootstrap</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5/dist/css/bootstrap.min.css" rel="stylesheet">
  </head>
  <body>
    <main class="container py-4">

```

```
<h1 class="mb-3">Witaj w Bootstrapie</h1>

<button class="btn btn-primary">Przycisk</button>

</main>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5/dist/js/bootstrap.bundle.min.js"></script>

</body>

</html>
```

W zwykłym HTML używa się atrybutu `class`. W React używa się `className`, bo `class` jest słowem z JavaScriptu. To najważniejsza różnica składniowa przy przenoszeniu przykładów z dokumentacji Bootstrapa do komponentów React.

## 2.2. CDN czy npm

| SPOSÓB                | KIEDY UŻYWAĆ                                | ZALETY                                               | OGRANICZENIA                                     |
|-----------------------|---------------------------------------------|------------------------------------------------------|--------------------------------------------------|
| CDN                   | prosta strona HTML, szybki prototyp         | bez instalacji, działa od razu                       | zależność od zewnętrznego adresu, mniej kontroli |
| npm                   | projekt React, Vite, CRA, większa aplikacja | wersja w <code>package.json</code> , import w kodzie | wymaga Node.js i bundlera                        |
| pobrane pliki lokalne | statyczna strona bez bundlera               | pełna kontrola nad plikami                           | ręczna aktualizacja wersji                       |

## 2.3. Bootstrap w pliku HTML bez JavaScriptu

Duża część Bootstrapa działa bez JavaScriptu: grid, przyciski, karty, formularze, tabele, kolory, spacing, flex, typografia. JavaScript jest potrzebny dopiero do komponentów takich jak modal, dropdown, collapse, offcanvas, tooltip, popover, carousel.

```
<section class="container py-5">
  <div class="row g-4">
    <article class="col-md-4">
      <div class="card h-100 shadow-sm">
        <div class="card-body">
          <h2 class="h5">Karta</h2>
          <p class="text-muted">Ten element nie wymaga JS Bootstrapa.</p>
          <a class="btn btn-primary" href="#">Czytaj więcej</a>
        </div>
      </div>
    </article>
  </div>
</section>
```

## 3. Responsywność i breakpointy

Bootstrap jest projektowany mobile-first. Oznacza to, że klasy bez breakpointu działają od najmniejszych ekranów, a klasy z breakpointem włączają się od określonej szerokości w górę. Przykład: `col-12 col-md-6 col-xl-3` oznacza pełną szerokość na telefonie, pół szerokości od `md` i ćwierć szerokości od `xl`.

3.1. Breakpointy

| BREAKPOINT | MINIMALNA SZEROKOŚĆ | PRZYKŁAD KLASY               |
|------------|---------------------|------------------------------|
| brak       | 0 px                | col-12, d-block, text-center |
| sm         | 576 px              | col-sm-6, d-sm-flex          |
| md         | 768 px              | col-md-4, text-md-start      |
| lg         | 992 px              | col-lg-3, d-lg-none          |
| xl         | 1200 px             | col-xl-2                     |
| xxl        | 1400 px             | container-xxl                |

3.2. Wzorce responsywnego układu

| CEL                                           | KLASY                                |
|-----------------------------------------------|--------------------------------------|
| Jedna kolumna na telefonie, dwie na desktopie | col-12 col-md-6                      |
| Karty 1/2/3/4 w zależności od szerokości      | col-12 col-sm-6 col-lg-4 col-xl-3    |
| Sidebar + treść                               | col-12 col-lg-3 oraz col-12 col-lg-9 |
| Formularz dwukolumnowy od md                  | row g-3 + col-md-6                   |
| Ukrycie elementu na telefonie                 | d-none d-md-block                    |
| Pokazanie elementu tylko na telefonie         | d-md-none                            |

3.3. Przykład: panel z bocznym menu

```
<main class="container-fluid py-4">
  <div class="row g-4">
    <aside class="col-12 col-lg-3">
      <nav class="list-group">
        <a class="list-group-item list-group-item-action active" href="#">Start</a>
        <a class="list-group-item list-group-item-action" href="#">Użytkownicy</a>
        <a class="list-group-item list-group-item-action" href="#">Ustawienia</a>
      </nav>
    </aside>
    <section class="col-12 col-lg-9">
      <div class="card shadow-sm">
        <div class="card-body">
          <h1 class="h3">Panel</h1>
          <p class="mb-0">Treść główna układu się pod menu na małych ekranach.</p>
        </div>
      </div>
    </section>
  </div>
</main>
```

4. Utilities — klasy pomocnicze w praktyce

Utilities to krótkie klasy, które rozwiązują pojedynczy problem: odstęp, kolor, układ flex, wyrównanie tekstu, cień, obramowanie, widoczność. To one sprawiają, że Bootstrap pozwala szybko składać interfejs bez pisania wielu własnych klas CSS.

4.1. Spacing

| KLASA   | ZNACZENIE                            |
|---------|--------------------------------------|
| m-3     | margin z każdej strony               |
| mt-4    | margin-top                           |
| mb-0    | brak marginesu dolnego               |
| mx-auto | automatyczne marginesy poziome       |
| p-3     | padding z każdej strony              |
| px-4    | padding lewo/prawo                   |
| py-5    | padding góra/dół                     |
| gap-3   | odstęp między elementami w flex/grid |

4.2. Display i flex

| KLASA                   | ZNACZENIE                            |
|-------------------------|--------------------------------------|
| d-flex                  | włącza flexbox                       |
| d-grid                  | włącza CSS grid dla prostych układów |
| d-none                  | ukrywa element                       |
| d-md-block              | pokazuje jako block od md            |
| justify-content-between | rozsusza elementy na boki            |
| align-items-center      | wyrównuje w osi poprzecznej          |
| flex-column             | układ pionowy                        |
| flex-wrap               | zawijanie elementów                  |

4.3. Typografia i kolory

| KLASA                   | ZNACZENIE                            |
|-------------------------|--------------------------------------|
| h1 ... h6               | wygląd nagłówka bez zmiany semantyki |
| display-1 ... display-6 | duże nagłówki ekspozycyjne           |
| lead                    | wyróżniony akapit                    |
| small                   | mniej tekst                          |
| fw-bold                 | pogrubienie                          |
| text-muted              | tekst pomocniczy                     |
| text-primary            | kolor tekstu primary                 |
| bg-light                | jasne tło                            |
| text-bg-success         | połączony kolor tekstu i tła         |



4.4. Obramowania, cienie i zaokrąglenia

| KLASA                       | ZNACZENIE         |
|-----------------------------|-------------------|
| <code>border</code>         | obramowanie       |
| <code>border-0</code>       | brak obramowania  |
| <code>border-primary</code> | kolor obramowania |
| <code>rounded</code>        | zaokrąglenie      |
| <code>rounded-circle</code> | koło              |
| <code>shadow-sm</code>      | mały cień         |
| <code>shadow</code>         | standardowy cień  |
| <code>shadow-none</code>    | brak cienia       |

4.5. Przykład karty z utilities

```
<article class="border rounded shadow-sm p-3 bg-white">
  <div class="d-flex justify-content-between align-items-start gap-3">
    <div>
      <h2 class="h5 mb-1">Raport miesięczny</h2>
      <p class="text-muted mb-0">Ostatnia aktualizacja: dzisiaj</p>
    </div>
    <span class="badge text-bg-success">Gotowe</span>
  </div>
</article>
```

5. Formularze i walidacja

Bootstrap daje gotowy wygląd pól formularza, ale nie wykonuje za aplikację logiki walidacji. Walidacja nadal należy do JavaScriptu, Reacta albo backendu. Bootstrap zapewnia klasy wizualne: `form-control`, `form-select`, `form-check`, `is-valid`, `is-invalid`, `valid-feedback`, `invalid-feedback`.

5.1. Podstawowy formularz

```
<form class="card shadow-sm">
  <div class="card-body">
    <div class="mb-3">
      <label for="email" class="form-label">E-mail</label>
      <input id="email" type="email" class="form-control" placeholder="jan@example.com">
      <div class="form-text">Nie udostępniamy adresu innym osobom.</div>
    </div>
    <div class="mb-3">
      <label for="role" class="form-label">Rola</label>
      <select id="role" class="form-select">
        <option>Użytkownik</option>
        <option>Administrator</option>
      </select>
    </div>
    <button class="btn btn-primary" type="submit">Zapisz</button>
```

```
    </div>
  </form>
```

## 5.2. Formularz w React z klasami Bootstrapa

```
function ContactForm() {
  const [email, setEmail] = useState("");
  const valid = email.includes("@");

  return (
    <form className="card shadow-sm" onSubmit={(e) => e.preventDefault()}>
      <div className="card-body">
        <label htmlFor="email" className="form-label">E-mail</label>
        <input
          id="email"
          type="email"
          className={`form-control ${email && !valid ? "is-invalid" : ""}`}
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        {email && !valid && <div className="invalid-feedback">Podaj poprawny e-mail.</div>}
        <button className="btn btn-primary mt-3" type="submit" disabled={!valid}>Zapisz</button>
      </div>
    </form>
  );
}
```

## 5.3. Input group

```
<div class="input-group mb-3">
  <span class="input-group-text">@</span>
  <input type="text" class="form-control" placeholder="nazwa użytkownika">
  <button class="btn btn-outline-secondary" type="button">Sprawdź</button>
</div>
```

## 6. Komponenty Bootstrapa — katalog praktyczny

Komponenty Bootstrapa to gotowe układy HTML z klasami. Niektóre są czysto CSS-owe, np. `card`, `alert`, `badge`, `list-group`. Inne wymagają JavaScriptu, np. `modal`, `dropdown`, `collapse`, `offcanvas`, `tooltip`.

| KOMPONENT    | KLUCZOWE KLASY          | CZY WYMAGA JS | TYPOWE UŻYCIE      |
|--------------|-------------------------|---------------|--------------------|
| Alert        | alert alert-primary     | nie           | komunikaty         |
| Badge        | badge text-bg-success   | nie           | statusy i liczniki |
| Breadcrumb   | breadcrumb              | nie           | ścieżka nawigacji  |
| Button group | btn-group               | nie           | grupa akcji        |
| Card         | card card-body          | nie           | karty treści       |
| Carousel     | carousel                | tak           | slajder            |
| Collapse     | collapse                | tak           | zwijane sekcje     |
| Dropdown     | dropdown-menu           | tak           | menu akcji         |
| List group   | list-group-item         | nie           | listy opcji        |
| Modal        | modal                   | tak           | okno dialogowe     |
| Navbar       | navbar navbar-expand-lg | częściowo     | nawigacja          |
| Offcanvas    | offcanvas               | tak           | panel boczny       |
| Pagination   | pagination page-item    | nie           | stronicowanie      |
| Progress     | progress progress-bar   | nie           | postęp             |
| Spinner      | spinner-border          | nie           | ładowanie          |
| Toast        | toast                   | tak           | krótkie komunikaty |
| Tooltip      | tooltip                 | tak           | podpowiedzi        |

6.1. Alerty

```
<div class="alert alert-success" role="alert">
  Dane zostały zapisane.
</div>
<div class="alert alert-danger" role="alert">
  Nie udało się pobrać danych.
</div>
```

6.2. Karty

```
<div class="card shadow-sm">
  <div class="card-header bg-body">Nagłówek</div>
  <div class="card-body">
    <h2 class="h5 card-title">Tytuł karty</h2>
    <p class="card-text">Opis elementu.</p>
    <a href="#" class="btn btn-primary">Akcja</a>
  </div>
</div>
```

### 6.3. Nawigacja

```
<nav class="navbar navbar-expand-lg bg-body-tertiary border-bottom">
  <div class="container">
    <a class="navbar-brand" href="#">Aplikacja</a>
    <div class="navbar-nav">
      <a class="nav-link active" href="#">Start</a>
      <a class="nav-link" href="#">Produkty</a>
      <a class="nav-link" href="#">Kontakt</a>
    </div>
  </div>
</nav>
```

## 7. JavaScript Bootstrapa i React

Bootstrap ma własne komponenty JavaScript sterowane atrybutami `data-bs-*`. React ma własny model stanu i renderowania. W prostych stronach HTML można używać `data-bs-*`, ale w React często czytelniej jest kontrolować widoczność komponentu przez `useState`.

### 7.1. Collapse przez Bootstrap JS

```
<button class="btn btn-primary" type="button" data-bs-toggle="collapse" data-bs-target="#opis">
  Pokaż opis
</button>
<div class="collapse mt-3" id="opis">
  <div class="card card-body">Treść zwijanej sekcji.</div>
</div>
```

### 7.2. Collapse kontrolowany Reactem

```
function CollapseExample() {
  const [open, setOpen] = useState(false);

  return (
    <section>
      <button className="btn btn-primary" type="button" onClick={() => setOpen((v) => !v)}>
        {open ? "Ukryj" : "Pokaż"} opis
      </button>
      {open && (
        <div className="card card-body mt-3">Treść zwijanej sekcji.</div>
      )}
    </section>
  );
}
```

W React podejście ze stanem jest często bardziej przewidywalne: widoczność zależy od `open`, a nie od ukrytego stanu komponentu Bootstrapa poza Reactem.

7.3. Kiedy importować bootstrap.bundle

| SYTUACJA                                   | IMPORT JS POTRZEBNY?         |
|--------------------------------------------|------------------------------|
| tylko grid, karty, formularze, utility     | nie                          |
| modal przez <code>data-bs-toggle</code>    | tak                          |
| dropdown przez <code>data-bs-toggle</code> | tak                          |
| tooltip/popover                            | tak, dodatkowo inicjalizacja |
| własny modal kontrolowany stanem React     | nie                          |

8. Motywy, kolory, CSS variables i Sass

Bootstrap ma gotowy system kolorów i wiele zmiennych CSS. Można używać domyślnego motywu, nadpisywać klasy własnym CSS albo zbudować własną wersję przez Sass. W małych projektach najczęściej wystarczy import Bootstrapa i kilka własnych klas po nim.

8.1. Kolory semantyczne

| NAZWA     | TYPOWE ZNACZENIE  | PRZYKŁADY KLAS                                                                 |
|-----------|-------------------|--------------------------------------------------------------------------------|
| primary   | główna akcja      | <code>btn-primary</code> , <code>text-primary</code> , <code>bg-primary</code> |
| secondary | akcja drugorzędna | <code>btn-secondary</code> , <code>text-secondary</code>                       |
| success   | sukces            | <code>alert-success</code> , <code>text-bg-success</code>                      |
| danger    | błąd lub usuwanie | <code>btn-danger</code> , <code>alert-danger</code>                            |
| warning   | ostrzeżenie       | <code>text-bg-warning</code>                                                   |
| info      | informacja        | <code>alert-info</code>                                                        |
| light     | jasne tło         | <code>bg-light</code>                                                          |
| dark      | ciemne tło        | <code>bg-dark text-white</code>                                                |

8.2. Nadpisywanie po imporcie

```
/* index.css importowany po bootstrap.css */
.app-shell {
  min-height: 100vh;
  background: #f7f8fa;
}

.card-dashboard {
  border: 0;
  box-shadow: 0 0.5rem 1rem rgba(0, 0, 0, 0.08);
}
```

8.3. CSS variables

```
:root {
  --bs-primary: #315efb;
  --bs-border-radius: 0.75rem;
}
```

```
.hero {  
  background: var(--bs-primary);  
  color: white;  
}
```

## 9. Dostępność i semantyka

Bootstrap zapewnia klasy, ale nie zastępuje poprawnego HTML. Przyciski powinny być przyciskami, linki linkami, pola formularzy powinny mieć etykiety, a komunikaty błędów powinny być powiązane z polami albo widoczne w miejscu, gdzie użytkownik ich oczekuje.

| ELEMENT          | ZASADA                                                                                            |
|------------------|---------------------------------------------------------------------------------------------------|
| Przycisk akcji   | użyj <code>&lt;button type="button"&gt;</code> albo <code>type="submit"</code> zgodnie z rolą     |
| Link nawigacyjny | użyj <code>&lt;a href&gt;</code> albo routerowego <code>Link</code> w React                       |
| Pole formularza  | połącz <code>label</code> z polem przez <code>for</code> / <code>htmlFor</code> i <code>id</code> |
| Alert błędu      | dodaj <code>role="alert"</code>                                                                   |
| Ikona bez tekstu | dodaj <code>aria-label</code> na przycisku albo tekst ukryty klasą <code>visually-hidden</code>   |
| Modal            | użyj <code>role="dialog"</code> i zadбай o zamykanie                                              |
| Kolor statusu    | dodaj tekst, nie opieraj się tylko na kolorze                                                     |

### 9.1. visually-hidden

```
<button class="btn btn-outline-danger" type="button">  
  <i class="bi bi-trash"></i>  
  <span class="visually-hidden">Usuń element</span>  
</button>
```

## 10. Złożone przykłady praktyczne

Poniższe przykłady są dłuższe i pokazują, jak klasy Bootstrapa łączyć w całe widoki. Większość z nich można wkleić jako JSX po zamianie `class` na `className`; wersje Reactowe są zapisane od razu z `className`.

### 10.1. Landing page usługi

```
function LandingPage() {  
  return (  
    <main>  
      <section className="bg-dark text-white py-5">  
        <div className="container py-5">  
          <div className="row align-items-center g-4">  
            <div className="col-lg-7">  
              <span className="badge text-bg-primary mb-3">Nowość</span>  
              <h1 className="display-4 fw-bold">Aplikacja do zarządzania zespołem</h1>  
              <p className="lead text-white-50">Zadania, raporty i komunikacja w jednym miejscu.</p>  
              <div className="d-flex gap-2 flex-wrap">  
                <a className="btn btn-primary btn-lg" href="#start">Rozpocznij</a>  

```

```

        <a className="btn btn-outline-light btn-lg" href="#features">Zobacz funkcje</a>
      </div>
    </div>
    <div className="col-lg-5">
      <div className="card text-dark shadow-lg">
        <div className="card-body p-4">
          <h2 className="h5">Status projektu</h2>
          <div className="progress mb-3"><div className="progress-bar" style={{ width: "72%" }}>72%</div></div>
          <p className="mb-0 text-muted">12 zadań zakończonych, 4 w toku.</p>
        </div>
      </div>
    </div>
  </div>
</section>
</main>
);
}

```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

## 10.2. Dashboard administracyjny

```

function Dashboard() {
  const stats = [
    { label: "Użytkownicy", value: 1280, color: "primary" },
    { label: "Zamówienia", value: 342, color: "success" },
    { label: "Błędy", value: 7, color: "danger" },
  ];

  return (
    <main className="container-fluid py-4">
      <div className="row g-3 mb-4">
        {stats.map((stat) => (
          <section className="col-md-4" key={stat.label}>
            <div className="card shadow-sm h-100">
              <div className="card-body">
                <p className="text-muted mb-1">{stat.label}</p>
                <h2 className={`text-${stat.color}`}>{stat.value}</h2>
              </div>
            </div>
          </section>
        ))}
      </div>
      <div className="card shadow-sm">
        <div className="card-header bg-body"><h1 className="h5 mb-0">Ostatnie zdarzenia</h1></div>
        <ul className="list-group list-group-flush">
          <li className="list-group-item d-flex justify-content-between"><span>Logowanie</span><span className="badge text-bg-success">0K</span></li>
          <li className="list-group-item d-flex justify-content-between"><span>Import danych</span><span className="badge text-bg-warning">W toku</span></li>
        </ul>
      </div>
    </main>
  );
}

```

```

    </div>
  </main>
);
}

```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

### 10.3. Formularz checkout

```

function CheckoutForm() {
  return (
    <main className="container py-4">
      <div className="row g-4">
        <section className="col-lg-8">
          <form className="card shadow-sm">
            <div className="card-body">
              <h1 className="h4 mb-3">Dane zamówienia</h1>
              <div className="row g-3">
                <div className="col-md-6"><label className="form-label">Imię</label><input className="form-control" /></div>
                <div className="col-md-6"><label className="form-label">Nazwisko</label><input className="form-control" /></div>
                <div className="col-12"><label className="form-label">Adres</label><input className="form-control" /></div>
                <div className="col-md-6"><label className="form-label">Miasto</label><input className="form-control" /></div>
                <div className="col-md-6"><label className="form-label">Kod pocztowy</label><input className="form-control" />
              </div>
            </div>
            <button className="btn btn-primary mt-4" type="submit">Złóż zamówienie</button>
          </form>
        </section>
        <aside className="col-lg-4">
          <div className="card shadow-sm"><div className="card-body"><h2 className="h5">Podsumowanie</h2><p>Razem: 249,00 zł</p>
          </div></div>
        </aside>
      </div>
    </main>
  );
}

```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

### 10.4. Katalog produktów z filtrami

```

function ProductCatalog({ products }) {
  return (
    <main className="container py-4">
      <div className="d-flex justify-content-between align-items-center mb-3">
        <h1 className="h3 mb-0">Produkty</h1>
        <select className="form-select w-auto"><option>Sortuj po nazwie</option></select>
      </div>
      <div className="row g-4">

```



```

<aside className="col-lg-3"><div className="card"><div className="card-body"><h2 className="h5">Filtry</h2><input
className="form-control" placeholder="Szukaj" /></div></div></aside>

<section className="col-lg-9">
  <div className="row g-3">
    {products.map((p) => (
      <article className="col-md-6 col-xl-4" key={p.id}>
        <div className="card h-100 shadow-sm"><div className="card-body"><h3 className="h5">{p.name}</h3><p
className="text-muted">{p.category}</p><button className="btn btn-primary">Dodaj</button></div></div>
        </article>
      )
    )}
  </div>
</section>
</div>
</main>
);
}

```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

### 10.5. Strona artykułów / portfolio

```

function BlogGrid({ posts }) {
  return (
    <main className="container py-5">
      <header className="text-center mb-5">
        <h1 className="display-6">Artykuły</h1>
        <p className="lead text-muted">Najnowsze materiały i poradniki.</p>
      </header>
      <div className="row g-4">
        {posts.map((post) => (
          <article className="col-md-6 col-lg-4" key={post.id}>
            <div className="card h-100 shadow-sm">
              <div className="card-body">
                <span className="badge text-bg-info mb-2">{post.category}</span>
                <h2 className="h5">{post.title}</h2>
                <p className="text-muted">{post.excerpt}</p>
              </div>
              <div className="card-footer bg-body"><a href="#" className="btn btn-outline-primary btn-sm">Czytaj</a></div>
            </div>
          </article>
        )
        )}
      </div>
    </main>
  );
}

```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

## 10.6. Widok ustawień aplikacji

```
function SettingsPage() {
  return (
    <main className="container py-4">
      <div className="row g-4">
        <aside className="col-lg-3">
          <div className="list-group">
            <a className="list-group-item list-group-item-action active" href="#">Profil</a>
            <a className="list-group-item list-group-item-action" href="#">Bezpieczeństwo</a>
            <a className="list-group-item list-group-item-action" href="#">Powiadomienia</a>
          </div>
        </aside>
        <section className="col-lg-9">
          <form className="card shadow-sm">
            <div className="card-body">
              <h1 className="h4">Ustawienia profilu</h1>
              <label className="form-label">Nazwa użytkownika</label>
              <input className="form-control mb-3" />
              <div className="form-check form-switch mb-3">
                <input className="form-check-input" type="checkbox" id="newsletter" />
                <label className="form-check-label" htmlFor="newsletter">Powiadomienia e-mail</label>
              </div>
              <button className="btn btn-primary" type="submit">Zapisz</button>
            </div>
          </form>
        </section>
      </div>
    </main>
  );
}
```

Ten przykład można potraktować jako gotowy układ bazowy. Najważniejsze są klasy siatki, odstępy `g-*`, karty, formularze i jasny podział na sekcje.

# 11. Szybkie tabele referencyjne

## 11.1. Najczęstsze klasy

| OBSZAR     | KLASY                                                                                                                   |
|------------|-------------------------------------------------------------------------------------------------------------------------|
| kontener   | <code>container</code> , <code>container-fluid</code>                                                                   |
| grid       | <code>row</code> , <code>col</code> , <code>col-md-6</code> , <code>g-3</code>                                          |
| spacing    | <code>m-*</code> , <code>mt-*</code> , <code>mb-*</code> , <code>p-*</code> , <code>px-*</code> , <code>py-*</code>     |
| flex       | <code>d-flex</code> , <code>justify-content-*</code> , <code>align-items-*</code> , <code>gap-*</code>                  |
| tekst      | <code>text-*</code> , <code>fw-bold</code> , <code>lead</code> , <code>small</code> , <code>h1</code> - <code>h6</code> |
| tło        | <code>bg-*</code> , <code>text-bg-*</code>                                                                              |
| przyciski  | <code>btn</code> , <code>btn-primary</code> , <code>btn-outline-*</code> , <code>btn-sm</code> , <code>btn-lg</code>    |
| formularze | <code>form-control</code> , <code>form-select</code> , <code>form-check</code> , <code>input-group</code>               |
| karty      | <code>card</code> , <code>card-body</code> , <code>card-header</code> , <code>card-footer</code>                        |
| tabele     | <code>table</code> , <code>table-striped</code> , <code>table-hover</code> , <code>table-responsive</code>              |

## 11.2. Częste błędy

| BŁĄD                                               | POPRAWKA                                                                |
|----------------------------------------------------|-------------------------------------------------------------------------|
| w React wpisano <code>class</code>                 | użyj <code>className</code>                                             |
| brak kontenera                                     | opakuj treść w <code>container</code> albo <code>container-fluid</code> |
| kolumny bez <code>row</code>                       | kolumny powinny być bezpośrednio w <code>.row</code>                    |
| za dużo własnego CSS                               | najpierw sprawdź utilities Bootstrapa                                   |
| modal przez data-bs w React bez JS bundle          | zaimportuj bundle albo kontroluj modal stanem Reacta                    |
| brak label w formularzu                            | dodaj <code>label</code> i <code>htmlFor</code> / <code>for</code>      |
| przycisk w formularzu wysyła formularz przypadkiem | dodaj <code>type="button"</code> dla zwykłej akcji                      |
| tabela nie mieści się na telefonie                 | opakuj w <code>table-responsive</code>                                  |